AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

RoadCracks

Autor:
Jakub Pyrdoł
Tymoteusz Zaczyk

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Cel projektu i rozwiązywany problem 3
1.2. Odbiorcy i inspiracja sposobem prezentacji danych 3
1.3. Podstawowy zakres funkcjonalny 3
1.4. Źródła danych wykorzystywane w aplikacji 4
1.5. Założenia dotyczące pomiaru i obsługi 5

2. Określenie wymagań szczegółowych **6**

2.1. Dane wejściowe i obsługa czujników 6
2.2. Wykrywanie zdarzeń i ocena odcinków drogi 6
2.3. Interfejs i obsługa aplikacji 7
2.4. Sytuacje niepożądane i wymagania jakościowe 8
2.5. Możliwości dalszego rozwoju 8

3. Projektowanie **10**

3.1. Przyjęte technologie i organizacja projektu 10
3.2. Struktura przygotowanego kodu 10
3.3. Nawigacja i podział na zakładki 11
3.4. Wstępny układ interfejsu 11
3.5. Motyw graficzny i dalsze projektowanie 12

4. Implementacja **13**

5. Testowanie **14**

6. Podręcznik użytkownika **15**

Literatura **16**

Spis rysunków **17**

Spis tabel **18**

Spis listingów **19**

1. Ogólne określenie wymagań

1.1. Cel projektu i rozwiązywany problem

Celem projektu jest stworzenie aplikacji mobilnej RoadCracks dla systemu Android, napisanej w języku Kotlin. Aplikacja będzie służyła do wykrywania i mapowania nierówności nawierzchni na podstawie danych rejestrowanych przez smartfon podczas jazdy. Głównym wynikiem jej działania będzie mapa z odcinkami dróg oznaczonymi kolorami odpowiadającymi oszacowanemu stopniowi ich uszkodzenia.

Projekt odpowiada na potrzebę uzyskania informacji o jakości nawierzchni na pokonywanej trasie. Pojedyncza informacja o wystąpieniu dziury nie opisuje całego odcinka drogi, dlatego RoadCracks ma uwzględniać zarówno nasilenie wstrząsów, jak i częstość ich występowania. Pozwoli to odróżnić odcinek z pojedynczą nierównością od drogi, na której niekorzystne zdarzenia powtarzają się na większej długości.

Ocena będzie miała charakter orientacyjny i będzie wynikała z pomiarów telefonu. Aplikacja nie będzie określała rzeczywistej głębokości ubytku, lecz poziom nasilenia zarejestrowanego zdarzenia. Dokładność wyniku będzie zależeć między innymi od prędkości przejazdu, właściwości pojazdu, mocowania urządzenia i jakości lokalizacji.

1.2. Odbiorcy i inspiracja sposobem prezentacji danych

RoadCracks będzie przeznaczona przede wszystkim dla kierowców, którzy chcą sprawdzić stan nawierzchni na pokonywanej trasie. Przejrzysta mapa powinna ułatwiać rozpoznanie miejsc, w których pomiary wskazują na większe nierówności, oraz odcinków, dla których nie zebrano jeszcze wystarczających danych.

Inspiracją dla sposobu prezentacji informacji jest aplikacja Yanosik [1], wykorzystująca mapę i informacje o zdarzeniach drogowych. W RoadCracks głównym przedmiotem prezentacji będzie jakość nawierzchni oraz położenie wykrytych nierówności i potencjalnych przeszkód.

1.3. Podstawowy zakres funkcjonalny

Docelowe działanie aplikacji będzie obejmowało cały przebieg pomiaru: od jego świadomego uruchomienia przez użytkownika, przez zbieranie danych, aż po zapis i prezentację wyniku. W podstawowym zakresie przewiduje się:

- rozpoczęcie i zakończenie rejestracji przejazdu;

- zapis przebiegu trasy i powiązanie wykrytych zdarzeń z lokalizacją;
- wykrywanie wstrząsów oraz oszacowanie ich nasilenia od lekkiego do krytycznego;
- oznaczanie potencjalnych przeszkód rozpoznanych na podstawie gwałtownego manewru omijania;
- ocenę odcinków drogi i przedstawienie wyniku za pomocą kolorów na mapie;
- dostosowanie jasnego lub ciemnego motywu do warunków oświetlenia.

Zdarzenia wynikające wyłącznie z gwałtownego manewru powinny być oznaczane oddzielnie od zdarzeń wykrytych na podstawie wstrząsu. Samo ominięcie przeszkody nie będzie traktowane jako potwierdzenie uszkodzenia nawierzchni. Odcinki bez wystarczających pomiarów powinny mieć odrębne oznaczenie, aby brak danych nie był interpretowany jako dobry stan drogi.

1.4. Źródła danych wykorzystywane w aplikacji

Projekt zakłada wykorzystanie czterech źródeł danych dostępnych w smartfonie. Są to moduł lokalizacji GPS oraz trzy czujniki telefonu:

- **GPS** umożliwi określenie położenia i prędkości, zapis trasy oraz przypisanie zdarzeniom współrzędnych geograficznych.
- **Akcelerometr** posłuży do wykrywania krótkotrwałych wstrząsów związanych z przejazdem przez nierówności. Analizie będzie podlegać przede wszystkim składowa przyspieszenia odpowiadająca kierunkowi pionowemu.
- **Żyroskop** dostarczy informacji o ruchu obrotowym telefonu. W połączeniu z prędkością z GPS umożliwi poszukiwanie wzorca gwałtownego manewru omijania przeszkody.
- **Czujnik światła** posłuży do automatycznego przełączania motywu interfejsu pomiędzy jasnym i ciemnym.

Dane ruchowe i lokalizacyjne powinny być łączone według czasu rejestracji, tak aby wykryte zdarzenie zostało przypisane do miejsca jego wystąpienia. Czujnik światła będzie wpływał na sposób prezentacji interfejsu, a nie na ocenę nawierzchni.

1.5. Założenia dotyczące pomiaru i obsługi

Podczas rejestracji telefon powinien być stabilnie zamocowany w uchwycie. Przed rozpoczęciem pomiaru aplikacja powinna ustalić jego położenie, aby analizować przyspieszenie w kierunku pionowym niezależnie od orientacji urządzenia. Poruszanie telefonem w ręce nie powinno być traktowane jako wiarygodny pomiar stanu nawierzchni.

Interfejs powinien być prosty i czytelny, z wyraźnym dostępem do mapy oraz funkcji rozpoczęcia i zakończenia pomiaru. Po uruchomieniu rejestracji zbieranie danych powinno odbywać się automatycznie. Szczegółowe wymagania funkcjonalne przedstawiono w rozdziale 2, a opis przygotowanego dotychczas prototypu i kierunek dalszego projektowania w rozdziale 3.

2. Określenie wymagań szczegółowych

2.1. Dane wejściowe i obsługa czujników

GPS. Aplikacja powinna odczytywać współrzędne geograficzne, prędkość, czas pomiaru oraz dostępną informację o dokładności lokalizacji. Dane te posłużą do rysowania trasy i umieszczania wykrytych zdarzeń na mapie. Wstrząs należy powiązać z odczytem lokalizacji odpowiadającym czasowi jego wystąpienia. Przy słabym sygnale aplikacja powinna sygnalizować obniżoną dokładność, zamiast przedstawiać położenie zdarzenia jako pewne.

Akcelerometr. Aplikacja powinna odczytywać przyspieszenie w trzech osiach i wyznaczać jego składową pionową. Analiza osi Z jako kierunku pionowego jest dopuszczalna tylko przy odpowiednim ustawieniu telefonu. Po uwzględnieniu orientacji urządzenia i składowej grawitacyjnej algorytm powinien wykrywać krótkie, wyraźne wstrząsy oraz nadawać im poziom nasilenia od lekkiego do krytycznego. Wstrząs może pochodzić od dziury, progu zwalniającego lub studzienki, dlatego sam pomiar nie powinien przesądzać o rodzaju obiektu.

Żyroskop. Aplikacja powinna analizować prędkość kątową telefonu w celu wykrywania sekwencji gwałtownego skrętu w jedną stronę i powrotu w przeciwną. Jeżeli taki wzorzec pojawi się podczas jazdy, przy odpowiedniej prędkości wskazanej przez GPS i bez istotnego wstrząsu, miejsce powinno zostać oznaczone jako potencjalna ominięta przeszkoda. Jest to przybliżone rozpoznanie manewru omijania, określanego jako manewr łosia. Żyroskop mierzy ruch telefonu, dlatego stabilne mocowanie urządzenia jest warunkiem wiarygodnej analizy.

Czujnik światła. Aplikacja powinna odczytywać natężenie oświetlenia i automatycznie włączać motyw ciemny przy słabym świetle, a jasny przy silnym oświetleniu. Zmiana powinna następować dopiero po utrzymaniu się warunków przez krótki czas, aby zapobiec częstemu przełączaniu motywu, na przykład podczas przejazdu pod drzewami. Użytkownik powinien mieć także możliwość ręcznego wyboru motywu.

2.2. Wykrywanie zdarzeń i ocena odcinków drogi

Pomiary z różnych źródeł powinny być łączone według czasu ich rejestracji. Przed rozpoczęciem przejazdu aplikacja powinna wykonać krótką kalibrację położenia telefonu. Algorytm powinien ograniczać wpływ drobnych drgań, ruchów telefonu i wielokrotnego wykrywania tego samego wstrząsu. Wykrywanie zdarzeń drogowych po-

winno działać podczas jazdy; odczyty z postoju nie powinny automatycznie tworzyć zgłoszeń.

Dla każdego zdarzenia należy zapisać jego czas, położenie, prędkość przejazdu, typ wykrycia i oszacowany poziom nasilenia. Zdarzenia należy przypisywać do odcinków trasy, uwzględniając dokładność GPS. Kolor odcinka powinien zależeć od liczby zdarzeń w odniesieniu do jego długości oraz ich siły. Potencjalny manewr ominięcia przeszkody nie powinien samodzielnie oznaczać krytycznego uszkodzenia nawierzchni.

Przyjętą skalę prezentacji stanu nawierzchni przedstawiono w tabeli 2.1. Poszczególne kolory odpowiadają oszacowanemu nasileniu nierówności, a kolor szary oznacza brak wystarczających danych. Progi wykrywania, granice poziomów nasilenia i sposób łączenia pomiarów zostaną dobrane podczas testów, ponieważ wynik zależy między innymi od prędkości, mocowania telefonu i właściwości pojazdu.

Tab. 2.1. Skala oceny odcinków drogi

Kolor	Znaczenie
Zielony	Brak istotnych nierówności w zebranych pomiarach.
Żółty	Niewielkie nierówności i lekkie wstrząsy.
Pomarańczowy	Nierówności o średnim nasileniu.
Czerwony	Poważne nierówności i silne wstrząsy.
Bordowy	Najwyższy poziom nasilenia, oceniony jako krytyczny.
Szary	Brak danych wystarczających do oceny odcinka.

Brak zgłoszeń na niezmierzonym odcinku nie może być utożsamiany z dobrym stanem nawierzchni. Potencjalne ominięte przeszkody powinny być prezentowane za pomocą osobnego znacznika, odróżniającego je od zdarzeń wykrytych na podstawie wstrząsu.

2.3. Interfejs i obsługa aplikacji

Główny ekran powinien zawierać mapę, bieżącą lokalizację użytkownika, pokonywaną trasę oraz kolorowe oznaczenia ocenionych odcinków. Widoczna legenda powinna wyjaśniać znaczenie kolorów i znaczników. Ekran powinien również pokazywać prędkość, stan pomiaru i dostępność lokalizacji.

- **Rozpocznij pomiar** sprawdza uprawnienia i czujniki, uruchamia kalibrację, a następnie rejestrację przejazdu.
- **Zakończ pomiar** zatrzymuje rejestrację i zapisuje przejazd oraz wykryte zdarzenia.
- **Moja lokalizacja** ustawia mapę na bieżącej pozycji użytkownika.
- **Ustawienia** otwiera wybór motywu i informacje o dostępności czujników.

Dotknięcie oznaczenia zdarzenia powinno wyświetlać jego typ, czas rejestracji i poziom nasilenia, a wybór odcinka drogi powinien pokazywać jego ocenę. Podczas aktywnego pomiaru aplikacja powinna automatycznie aktualizować trasę i oznaczenia oraz zmieniać motyw zgodnie z ustawieniami. Komunikaty o problemach powinny być krótkie i jednoznaczne.

2.4. Sytuacje niepożądane i wymagania jakościowe

Przed rozpoczęciem pomiaru aplikacja powinna poprosić o dostęp do lokalizacji i wyjaśnić jego cel. Odmowa uprawnienia lub wyłączenie lokalizacji powinny skutkować komunikatem o braku możliwości przypisania zdarzeń do mapy. Przy chwilowej utracie GPS aplikacja nie powinna przypisywać nowych zdarzeń do ostatniej znanej pozycji jako aktualnej.

Brak akcelerometru powinien wyłączać wykrywanie wstrząsów, a brak żyroskopu wykrywanie manewrów omijania. Brak czujnika światła powinien pozostawiać możliwość ręcznego wyboru motywu. Ograniczona funkcjonalność musi być widoczna dla użytkownika. W razie utraty połączenia z internetem pomiary powinny być przechowywane lokalnie; jeżeli nie uda się pobrać mapy, aplikacja powinna poinformować o ograniczeniu jej widoku.

Interfejs powinien pozostawać płynny podczas zbierania danych, a pomiary powinny być zapisywane na bieżąco, aby ograniczyć ich utratę po przerwaniu pracy aplikacji. Odczyt czujników należy zakończyć po zakończeniu pomiaru, ograniczając zużycie baterii. Dostęp do danych o lokalizacji powinien być ograniczony do funkcji aplikacji, a rejestrowanie trasy powinno odbywać się po świadomym uruchomieniu pomiaru przez użytkownika.

2.5. Możliwości dalszego rozwoju

W kolejnych wersjach możliwe będzie współdzielenie zgłoszeń między użytkownikami, potwierdzanie nierówności na podstawie wielu przejazdów, uwzględnianie wieku

pomiarów oraz proponowanie tras o lepszej nawierzchni. Funkcje te stanowią kierunki rozwoju i nie są wymagane w podstawowym zakresie projektu.

3. Projektowanie

3.1. Przyjęte technologie i organizacja projektu

Dotychczas przygotowano wstępny prototyp interfejsu RoadCracks w języku Kotlin. Projekt ma strukturę aplikacji Android z modułem `app`, a jego konfigurację zapisano w plikach Gradle z rozszerzeniem `.kts`. W konfiguracji modułu aplikacji określono minimalny poziom API jako 24. Zgodność działania na poszczególnych urządzeniach będzie wymagała sprawdzenia podczas testowania.

Do budowy interfejsu wykorzystano Jetpack Compose [2] oraz komponenty Material 3. Widoki są opisane jako funkcje oznaczone adnotacją `@Composable`. Takie rozwiązanie pozwala rozwijać poszczególne ekrany jako osobne elementy, zachowując wspólny układ aplikacji i motyw graficzny. Plik z wersjami zależności znajduje się w katalogu `gradle`, a konfiguracja modułu wskazuje biblioteki Compose, Material 3, ikon oraz podglądu interfejsu.

Na obecnym etapie projekt koncentruje się na przygotowaniu nawigacji i prostego interfejsu. Rozbudowa zawartości zakładek oraz integracja danych pomiarowych będą wykonywane w kolejnych etapach.

3.2. Struktura przygotowanego kodu

Głównym punktem wejścia jest klasa `MainActivity`, dziedzicząca po `ComponentActivity`. W metodzie `onCreate` wywoływane jest `setContent`, które uruchamia interfejs Compose. Zawartość aplikacji jest otoczona funkcją `RoadCracksTheme`, a następnie przekazywana do głównego układu `RoadCracksMainLayout`.

W aktualnej wersji najważniejsze elementy projektu są rozmieszczone następująco:

- `MainActivity.kt` zawiera klasę startową, główny układ oraz funkcje odpowiadające poszczególnym ekranom;
- katalog `ui/theme` zawiera definicje kolorów, typografii i motywu aplikacji;
- `AndroidManifest.xml` określa podstawową konfigurację aplikacji i aktywność uruchamianą po jej otwarciu;
- pliki `build.gradle.kts` oraz katalog `gradle` zawierają konfigurację budowania i zależności.

Funkcje ekranów są już wydzielone logicznie, choć znajdują się w jednym pliku. Przy rozbudowie projektu można przenieść je do osobnych plików, a obsługę pomiarów oddzielić od kodu odpowiedzialnego za wyświetlanie interfejsu. Jest to plan dalszego uporządkowania kodu.

3.3. Nawigacja i podział na zakładki

Główny układ wykorzystuje komponent `Scaffold`, którego dolny obszar zawiera pasek `NavigationBar`. Poszczególne pozycje utworzono za pomocą `NavigationBarItem` [3]. Przyjęto pięć zakładek:

- **Główna** odpowiada funkcji `HomeScreen` i stanowi miejsce przeznaczone na podsumowanie aktywności;
- **Mapa** odpowiada funkcji `MapScreen` i będzie głównym miejscem prezentacji trasy oraz stanu nawierzchni;
- **Mapuj** odpowiada funkcji `MapujScreen` i jest przeznaczona do obsługi rejestrowania przejazdu;
- **Alert** odpowiada funkcji `AlertScreen` i stanowi miejsce przewidziane dla ostrzeżeń;
- **Profil** odpowiada funkcji `ProfileScreen` i jest miejscem przeznaczonym na ustawienia oraz zapisane informacje użytkownika.

Wybrana zakładka jest przechowywana w zmiennej `selectedItem`, której wartość początkowa wynosi 1, dlatego po uruchomieniu wyświetlany jest ekran mapy. Naciśnięcie pozycji paska zmienia tę wartość, a instrukcja `when` wybiera właściwą funkcję ekranu. Obecne przełączanie zakładek jest zrealizowane bez osobnego grafu nawigacji.

3.4. Wstępny układ interfejsu

Przyjęto prosty interfejs oparty na ikonach, krótkich etykietach i wspólnym dolnym pasku zakładek. Wstępne ekrany określają przeznaczenie poszczególnych części aplikacji. Ich szczegółowy wygląd, układ elementów oraz docelowa zawartość będą rozwijane w kolejnym etapie prac.

Na ekranie mapy przygotowano zastępczy obszar z napisem wskazującym miejsce przyszłej mapy. Nad dolnym paskiem nawigacji znajduje się pole wyszukiwania adresu. Wprowadzany tekst jest przechowywany w zmiennej `searchQuery`; przygotowano również przycisk jego wyczyszczenia. Wyszukiwanie nie jest jeszcze połączone z usługą adresową.

Po obu stronach obszaru mapy umieszczono wstępne panele z ikonami oraz opisami odległości i prędkości. Wartość odległości jest przykładowa, a oznaczenie prędkości nie przedstawia odczytu GPS. Kliknięcie paneli wyświetla krótkie komunikaty typu `Toast`, co pozwala sprawdzić reakcję interfejsu na dotyk.

Zakładka Mapuj zawiera przycisk rozpoczęcia nagrywania trasy, stanowiący na tym etapie element układu ekranu. Pozostałe zakładki mają proste widoki z ikoną, tytułem i tekstem opisującym ich przyszłą zawartość. Pozwala to rozwijać wygląd poszczególnych ekranów bez zmiany przyjętego podziału aplikacji.

3.5. Motyw graficzny i dalsze projektowanie

Funkcja `RoadCracksTheme` zawiera jasny i ciemny zestaw kolorów, a wybór motywu jest obecnie powiązany z ustawieniem systemowym. W kodzie uwzględniono również dynamiczne kolory na obsługiwanych wersjach systemu. Docelowe przełączanie motywu na podstawie czujnika światła, opisane w wymaganiach, pozostaje do przygotowania.

Kolejne prace obejmą dopracowanie zawartości zakładek, wybór i podłączenie mapy, obsługę uprawnień lokalizacji oraz połączenie interfejsu z modułem pomiarów. Następnie konieczne będzie zaprojektowanie zapisu danych i algorytmu łączącego wstrząsy z pozycją oraz oceniającego odcinki drogi. W przekazanej wersji kodu nie ma jeszcze obsługi GPS i czujników, rzeczywistego zapisu przejazdu ani klasyfikacji nierówności. Rozdział opisuje więc istniejący szkielet aplikacji oraz kierunek jego dalszej rozbudowy.

4. Implementacja

5. Testowanie

6. Podręcznik użytkownika

Literatura

[1] Yanosik. *Yanosik – system ostrzeżeń drogowych*. URL: https://yanosik.pl/ (dostęp 28.10.2026).

[2] Android Developers. *Jetpack Compose*. URL: https://developer.android.com/compose (dostęp 28.10.2026).

[3] Android Developers. *Navigation bar*. URL: https://developer.android.com/develop/ui/compose/components/navigation-bar (dostęp 28.10.2026).

Spis rysunków

Spis tabel

2.1. Skala oceny odcinków drogi 7

Spis listingów